Lab 3: async, the Chat Completions format, tokens, and unified models

Concepts we need before writing the provider adapters and `/v1/chat/completions`. No code is written in this lab.

---

1. `async` and `await`, and why a gateway needs them

The problem: waiting

When the gateway calls OpenAI, it sends a request and then **waits**, often 1 to 30 seconds, for the answer. During that wait our code does nothing at all, because the work happens on OpenAI's computers.

With normal ("synchronous") code, a worker that is waiting is **blocked**: it cannot do anything else until the answer arrives.

Real-life example: a waiter

- **Synchronous waiter:** takes table 1's order, walks to the kitchen, and *stands there* until the food is ready. Tables 2 to 10 wait. Ten tables take ten times as long.
- **Asynchronous waiter:** takes table 1's order, gives it to the kitchen, and *while the kitchen cooks* takes orders from tables 2, 3, 4... When a dish is ready, they serve it. One waiter handles many tables, because most of the time is spent waiting for the kitchen, not walking.

What the words mean

- `async def` marks a function that is allowed to pause while it waits. Calling it gives you a "to-do" (a *coroutine*); it starts running when you `await` it.
- `await` means "start this slow thing, and while it is waiting, let other work run. Come back here when it is done."
- The **event loop** is the "waiter" that keeps track of all the paused tasks and resumes each one when its answer arrives. FastAPI and Uvicorn run the event loop for you.

```
import asyncio

async def call_provider(name: str) -> str:
    await asyncio.sleep(2)          # stands in for "waiting for the provider's answer"
    return f"answer from {name}"

async def main() -> None:
    # Both calls wait at the same time, so this takes about 2 seconds, not 4.
    results = await asyncio.gather(call_provider("openai"), call_provider("anthropic"))
    print(results)

asyncio.run(main())
```

Why a gateway needs it

A gateway does almost no computing of its own; it is mostly **waiting on providers**. With `async`, one worker can have hundreds of provider calls "in the air" at once. Without it, each waiting request would occupy a whole worker, and a few slow requests would block everyone.

This is also why the spec asks for **`httpx` (async)**: it has an `AsyncClient` whose `await client.post(...)` lets other requests run while it waits.

One rule to remember: inside an `async def`, never call slow code that is *not* async (for example `time.sleep(5)` or the non-async `requests` library). It blocks the whole event loop, and every request on that worker freezes.

> Our `/health` and `/ready` functions use plain `def`. That is fine: FastAPI runs plain `def` endpoints in a separate thread so they cannot block the loop. Endpoints that call providers will be `async def`.

---

2. The OpenAI Chat Completions format

Our gateway speaks this format, so existing agent tools work with it unchanged.

The request

You send the **whole conversation** every time. The model remembers nothing between calls.

```
POST /v1/chat/completions
{
  "model": "auto",
  "messages": [
    { "role": "system",    "content": "You are a helpful assistant. Answer briefly." },
    { "role": "user",      "content": "What is the capital of France?" },
    { "role": "assistant", "content": "Paris." },
    { "role": "user",      "content": "And of Japan?" }
  ],
  "temperature": 0.2,
  "max_tokens": 50
}
```

- **`model`**: which model to use. In our gateway it is either a real model name or `"auto"` (let the router decide).
- **`messages`**: the conversation, oldest first. Each message has a **role**:

Role	Who is speaking	Example
`system`	The developer, setting rules and personality	"You are a helpful assistant."
`user`	The human (or the agent) asking	"What is the capital of France?"
`assistant`	Earlier replies from the model	"Paris."
`tool`	The result of a tool the model asked to call	`{"temp_c": 21}`

- **`temperature`**: randomness. `0` gives the most predictable answer; higher values are more creative.
- **`max_tokens`**: the most tokens the model may write in its answer (see section 3).

The response

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1761650000,
  "model": "gpt-4o-mini",
  "choices": [
    {
      "index": 0,
      "message": { "role": "assistant", "content": "Tokyo." },
      "finish_reason": "stop"
    }
  ],
  "usage": { "prompt_tokens": 42, "completion_tokens": 3, "total_tokens": 45 }
}
```

- `id`, `created` (a Unix timestamp), and `model` (the model that actually answered).
- `choices`: a *list*, because you can ask for several alternative answers (the `n` parameter). Almost always there is one, and you read `choices[0].message.content`.
- `finish_reason`: why the model stopped:
- `"stop"`: it finished naturally.
- `"length"`: it hit `max_tokens`, so the answer was cut off.
- `"tool_calls"`: it wants to call a tool.
- `usage`: how many tokens went in and came out. This is what you are billed for.

Our gateway will return exactly this shape **plus** an extra `gateway` object (tier, cost, fallback used...), as described in the spec. Clients that do not know about `gateway` simply ignore it.

---

3. Tokens: input and output

What a token is

LLMs do not read letters or whole words; they read **tokens**, which are small chunks of text. A rough rule for English: **1 token ≈ 4 characters ≈ ¾ of a word**.

```
"Hello, world!"        → "Hello" "," " world" "!"            ≈ 4 tokens
"unbelievable"         → "un" "believ" "able"                ≈ 3 tokens
```

The exact split depends on the model's *tokenizer*, so different providers count slightly differently for the same text. Code, numbers and non-English text often use more tokens per word.

Input tokens vs output tokens

- **Input tokens** (OpenAI calls them `prompt_tokens`): everything you **send**. That is the system message, the whole conversation history, tool definitions, and the new question.
- **Output tokens** (OpenAI calls them `completion_tokens`): everything the model **writes back**.

Why they matter

1. **Money.** Providers charge per million tokens, with *separate* prices for input and output. Output is usually several times more expensive than input. Example, with **made-up** prices of $1 per 1M input and $4 per 1M output tokens: `42 input + 3 output = 42 × $1/1M + 3 × $4/1M = $0.000042 + $0.000012 = $0.000054`. Our cost tracker (Feature 4) does exactly this, with `Decimal`.
2. **Limits.** Each model has a **context window**, the most tokens (input + output) it can handle in one call. That is why our config has `max_context_tokens` for each model.
3. **Long chats get expensive.** The full history is sent every time, so input tokens grow with each turn of the conversation.
4. **Routing.** Token count is one of the signals our complexity scorer uses.

---

4. Why the gateway uses its own "unified" models

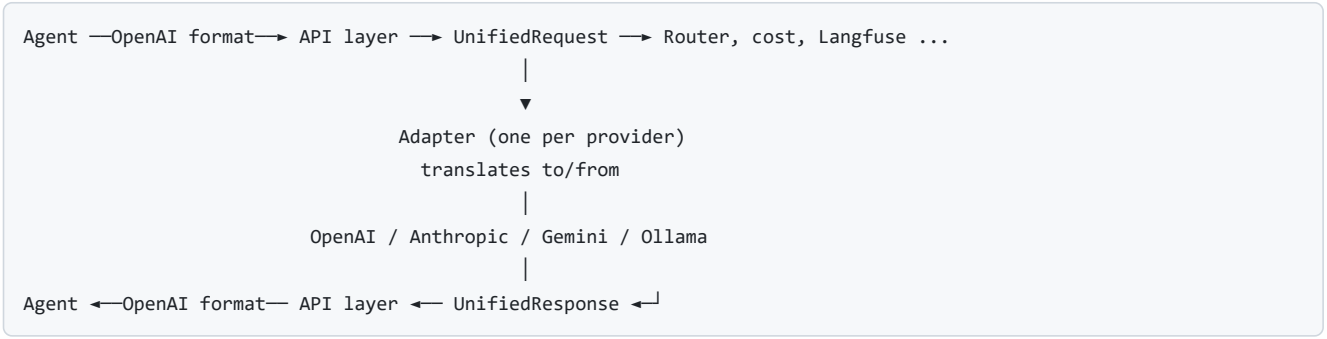
The problem: every provider speaks a different dialect

The same idea, "send messages, get an answer and token counts", looks different everywhere:

	OpenAI	Anthropic	Gemini	Ollama
System prompt	a message with role `system`	separate top-level `system` field	separate `systemInstruction` field	a message with role `system`
Messages field	`messages`	`messages`	`contents` (made of `parts`)	`messages`
Assistant role name	`assistant`	`assistant`	`model`	`assistant`
Answer text	`choices[0].message.content`	`content` (a list of blocks)	`candidates[0].content.parts`	`message.content`
Input tokens	`usage.prompt_tokens`	`usage.input_tokens`	`usageMetadata.promptTokenCount`	`prompt_eval_count`
Output tokens	`usage.completion_tokens`	`usage.output_tokens`	`usageMetadata.candidatesTokenCount`	`eval_count`
"Finished normally"	`stop`	`end_turn`	`STOP`	`done: true`

(Field names as documented by each provider today. The adapters will be checked against their current docs when we build them.)

The solution: one internal language, plus a translator per provider



The gateway defines its own `UnifiedRequest` and `UnifiedResponse` (Pydantic models). Only the **adapters** know about provider formats. Everything else (routing, retries, cost tracking, Langfuse) speaks just one language.

Why this is better than passing provider formats straight through

1. **The core code is written once.** The cost tracker reads `response.input_tokens` and never needs "if Anthropic... elif Gemini...".
2. **Fallback becomes possible.** If OpenAI fails, the *same* unified request goes to Anthropic's adapter. With pass-through, we would have to rewrite the request by hand at the moment of failure.
3. **Adding a provider is easy:** one new adapter class plus one config entry, and no other code changes (a spec requirement).
4. **Errors are normalized too.** Each provider's "too many requests" becomes our `RateLimitError`, so retry logic has one thing to check.
5. **Validation and safety.** Pydantic checks every request at the edge, and provider quirks cannot leak into the rest of the code.
6. **We are not tied to one vendor.** Our *external* API looks like OpenAI's, but our *internal* models are our own. If a provider changes its format, only its adapter changes.

Real-life example: a UN meeting. Each delegate speaks their own language, but every speech is interpreted into one shared working language. The organisers (our router, cost tracker and logs) only need to understand that one language, and adding a new country means hiring one interpreter (an adapter), not retraining everyone.

Quick check

Q1. The gateway has one worker and uses *synchronous* code. Ten agents send requests at the same moment, and each provider call takes 5 seconds. Roughly how long does the last agent wait? How long with `async`?

> **Answer:** About 50 seconds with synchronous code, because the requests are handled one after another (10 × 5 s). With `async`, about 5 seconds: all ten calls wait at the same time, and the worker handles each answer as it arrives.

Q2. A call sends a 50-token system message and a 30-token user question, and the model replies with 100 tokens. What are the input and output tokens? The user then asks a 20-token follow-up. What are the input tokens of the second call, and why?

> **Answer:** First call: 80 input tokens (50 + 30) and 100 output tokens. Second call: about 200 input tokens (50 + 30 + 100 + 20), because the whole conversation, including the model's earlier answer, is sent again every time. The model remembers nothing between calls.